

# ENGR 102 Summer Bridge

## Day 10 Student Workbook

Lectures 1-5 Recap, Direct Input, and Repetition Launch

Friday, July 17, 2026 • 50 points

|      |  |     |  |         |  |
|------|--|-----|--|---------|--|
| Name |  | UIN |  | Section |  |
|------|--|-----|--|---------|--|

**Boolean-operator convention.** Python operators appear as **and**, **or**, and **not**. Their lowercase spelling is required in code.

### Daily target

### increasing independence

Connect input, state, Boolean decisions, and testing to a first complete while-loop program.

|                         |                                                                                                                                                                                |
|-------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Allowed tools</b>    | input, int, float, arithmetic, comparisons, Boolean operators <b>and</b> , <b>or</b> , and <b>not</b> , if/elif/else, assignment, print, and a first while loop after the quiz |
| <b>Support supplied</b> | Variable names, a floor-validity rule, the movement goal, and named boundary tests are supplied.                                                                               |
| <b>You must decide</b>  | The conversion, validation branch order, continuation condition, direction decision, and one-floor update.                                                                     |
| <b>Scaffold level</b>   | Planning frame supplied; the complete elevator loop is student-authored.                                                                                                       |

### Scoring and completion standard

### 50 points

| Evidence               | Points | Secure work shows                                                   |
|------------------------|--------|---------------------------------------------------------------------|
| Integrated trace       | 14     | intermediate state, condition results, and exact output             |
| Diagnosis and repair   | 12     | actual, intended, cause, repair, and a boundary test when requested |
| Full program and tests | 24     | coherent executable logic, required output, and defended tests      |

**Independence standard.** You may use the supplied requirements and examples, but you must produce one connected solution. A collection of disconnected correct lines is not yet a complete program.

Begin with the trace, then use its evidence habits when you construct.

**Task 1. Integrated trace****14 points**

Trace types, Boolean subexpressions, and branch order. Do not jump directly to the final line.

```
sample = "14"
offset = 5
value = int(sample) * 2 + offset
stable = 20 <= value <= 35 and value % 3 == 0

if not stable:
    status = "review"
elif value >= 30:
    status = "high"
else:
    status = "normal"

print(value, stable, status)
```

**Your task**

1. Compute value and identify its type. Then evaluate each side of the `and` expression used to compute stable.
2. State which branch assigns status and explain why the earlier branch does not execute.
3. Write the exact printed output, including the Boolean capitalization.

**Task 2. Diagnose and repair****12 points**

The requirement is: approval requires training, and it also requires either a temperature from 15 through 30 inclusive or a supervisor override.

```
trained = False
temperature = 20
supervisor = False

approved = trained and temperature >= 15 or temperature <= 30 or supervisor
print(approved)
```

**Your task**

1. Show the actual Boolean result for the supplied values and explain why it violates the requirement.
2. Write one corrected Boolean assignment and defend its grouping with a test in which trained is False.

**Task 3. Construct a complete program****24 points across Pages 4-5****Program specification**

- Read `current_floor` and `desired_floor` as integers.
- Valid floors are 1 through 10 inclusive. Print invalid if either input is outside that interval.
- If the valid floors are equal, print already there.
- Otherwise, use a while loop that changes `current_floor` by exactly one floor per pass until it equals `desired_floor`.
- Print each newly reached floor inside the loop. Do not jump directly to the destination.

**Required planning evidence**

1. Write the invalid-input Boolean expression and the branch order that protects the loop.
2. Write the continuation condition and both possible one-floor updates before coding.

**Program workspace – begin the connected solution here.**

**Task 3 continued. Test and defend the program****included in 24 points**

Continue code if needed.

---

---

---

---

---

---

---

---

---

---

Required tests, expected results, and brief defense

|  |
|--|
|  |
|--|

# VIP Lab Alignment

## Bridge Day 10 • Contract 2d4127aac856

This page adds a current lab handoff while preserving the workbook's independence-ramp tasks.

### Why this page is here

**VIP Lab Day(s):** 5

**Activities:** 18.19, 18.21, 18.22

Begin with the notebook's required read-convert-compute-format-print reconnect, then transfer the elevator loop's continuation, update, and termination evidence into three while-loop activities.

### Open these current notebook cells

| Activity / stable cells                                  | Notebook work                | Evidence to carry forward                         |
|----------------------------------------------------------|------------------------------|---------------------------------------------------|
| <b>18.19</b><br>18-19-work / 18-19-transfer / 18-19-cold | <b>Countdown To Multiple</b> | Write a while condition from a divisibility goal. |
| <b>18.21</b><br>18-21-work / 18-21-transfer / 18-21-cold | <b>Cooldown Sequence</b>     | Track a current value and a stored sequence.      |
| <b>18.22</b><br>18-22-work / 18-22-transfer / 18-22-cold | <b>Rounds To Clear Queue</b> | Model repeated queue reduction.                   |

### Readiness check before you code

1. Write the five-part program shell, then for each mapped activity identify the input conversion, changing state, and continuation condition. \_\_\_\_\_

\_\_\_\_\_

2. Explain in one sentence why the update moves the state toward termination. \_\_\_\_\_

\_\_\_\_\_

### Notebook and submission procedure

1. Use the sample and transfer cells for formative practice; attempt before opening a reveal.
2. Complete the end-of-notebook Independent Program Studio without a reveal or copied solution. Only the Studio cells listed above are scored for independent mastery in the matching Lab Day notebook.
3. Save or download the completed notebook as one `.ipynb` file.
4. Upload that one notebook to the matching Gradescope assignment. Do not export a `.py` file or create a ZIP.